

Programming Project 3

● Graded

2 Hours Late

Student

Jinku Jung

Total Points

56 / 70 pts

Autograder Score

24.0 / 30.0

Failed Tests

Test Case 1 for correct Game.java (0/3)

Test Case 2 for correct Game.java (0/3)

Passed Tests

Everything compiles (21/21)

Test Case 3 for correct Game.java function (3/3)

Question 2

Autograder Adjustment

0 / 0 pts

✓ + 0 pts Autograder Correct

Question 3

Runtime

12 / 12 pts

✓ + 3 pts manual game allows for interactive play

✓ + 3 pts manual game determines and calculates winnings properly

✓ + 3 pts simulation determines winning strategy 0.56-0.6

✓ + 3 pts README explains how they determined the winning threshold value (explanation does not necessarily need to be correct)

💬 Amazing!

Question 4

Style

9 / 10 pts

Header

✓ + 2 pts complete header with Name, UNI, file description

Comments

✓ + 3 pts descriptive and concise comments throughout code

✓ + 1 pt descriptive variable names

line length

✓ + 1 pt 1-4 lines over 80 characters

✓ + 1 pt good use of white space

✓ + 1 pt proper indentation

🗨 Nice work!
(Try to keep your comments more concise in Simulation, leave the conceptual explanation in the readme!)
A few lines over 80 characters

Question 5

Design

11 / 18 pts

Method use

✓ + 2 pts Input and output only happen in Game.java class or main method

✓ + 1 pt Methods are all either accessors or mutators, but not both

Bells and whistles

✓ + 2 pts Interactive game allows multiple rounds tracking tokens across rounds

✓ + 2 pts Simulation class uses nested loops to create table

✓ + 2 pts In the table above they only track the min across each row, the max min for the table and the best strategy

✓ + 2 pts Logic behind explanation in readme of how threshold was determined is CORRECT

🗨 Nice work! When declaring instance variables, think about what information is necessary throughout the class versus what can be declared as a local variable within a certain method. Also, make sure your instance variables are private; your ComputerPlayers, gamesWon, etc. should not be accessible by other classes. Computer player tokenBalance should be handled in ComputerPlayer rather than Game; this is excess information that Game shouldn't need to worry about.

Question 6

README and Formatting

0 / 0 pts

✓ - 0 pts Includes README

Autograder Results

Everything compiles (21/21)

Test Case 1 for correct Game.java (0/3)

Test Failed: Assertion failed at edu.columbia.pp3.OddEvenTests.test_game1:17 (OddEvenTests.java)

Test Case 2 for correct Game.java (0/3)

Test Failed: Assertion failed at edu.columbia.pp3.OddEvenTests.test_game2:27 (OddEvenTests.java)

Test Case 3 for correct Game.java function (3/3)

Submitted Files

```
1  /* Name: Jinku Jung
2   * UNI: dj2598
3   * Assignment: Programming Project #3
4   * Date: 2/27/2021
5   */
6
7  /* File: Coin.java
8   *
9   * Purpose: This is the Coin class file for the Odd/Even game.
10  * (Adapted from Professor Cannon's Lecture code.)
11  */
12
13
14  /*****
15   * Coin.java
16   *
17   * Models a coin
18   *
19   *****/
20
21  public class Coin
22  {
23      // constants to remember encoding scheme
24      public final int HEADS = 0;
25      public final int TAILS = 1;
26
27      // only one instance variable
28      private int face;
29
30      // flip the coin to initialize it
31      public Coin()
32      {
33          flip();
34      }
35
36      // mutator that flips the coin
37      public void flip()
38      {
39          face = (int) (Math.random() * 2);
40      }
41
42      // accessor for to see if coin is heads
43      public int tossResult()
44      {
45          return face;
46      }
47
48  }
```

```
1  /* Name: Jinku Jung
2  * UNI: dj2598
3  * Assignment: Programming Project #3
4  * Date: 2/27/2021
5  */
6
7  /* File: ComputerPlayer.java
8  * Purpose: This is the ComputerPlayer class file of the Odd/Even game.
9  * It represents a single instance of a CPU-based player in the game.
10 */
11
12 public class ComputerPlayer {
13
14     //Instance variables for the ComputerPlayer class
15
16     public double t;
17     public int TokenBalance;
18     public int roundMove;
19     public int gamesWon;
20
21     //Constructor for the ComputerPlayer class
22
23     public ComputerPlayer(double threshold) {
24
25         t=threshold;
26         TokenBalance = 0;
27         gamesWon = 0;
28         roundMove = 0;
29
30     }
31
32     /* Method: getRoundMove()--This method gets the actual number of
33     fingers to be thrown for the current Odd/Even game round, by
34     the calling player object. */
35
36     public double getRoundMove() {
37
38         return this.roundMove;
39
40     }
41
42     /* Method: getGamesWon()--This method gets the actual number
43     of games won, as of now, by the calling player object. */
44
45     public int getGamesWon() {
46
47         return this.gamesWon;
48
49     }
```

```
50
51  /* Method: setGamesWon()--This method increments the variable
52  tracking the number of games won, by the calling player object,
53  by one game. */
54
55  public void setGamesWon() {
56      this.gamesWon++;
57  }
58 }
```

```
1  /* Name: Jinku Jung
2   * UNI: dj2598
3   * Assignment: Programming Project #3
4   * Date: 2/27/2021
5   */
6
7  /* File: Game.java
8   *
9   * Purpose: This is the main Game class file for
10  * the Odd/Even game.
11  */
12
13  import java.util.Scanner;
14
15  public class Game {
16
17      //Instance Variables for the Game class:
18
19      public ComputerPlayer p1;
20      public ComputerPlayer p2;
21      private int p1TokenBalance;
22      private int p2TokenBalance;
23      private int CPUTokenBalance;
24      public Coin myCoin;
25      public int myRoundMove;
26      public int CPURoundMove;
27      public int gamesWon;
28      private Scanner input;
29      private int roundSum;
30      private int isPlaying;
31      public double t1;
32      public double t2;
33
34      /* Constructor: Game()--This version of the Odd/Even
35       * game constructor is for Part 1 of PP #3: it
36       * takes in ZERO input parameters */
37
38      public Game() {
39
40          p1TokenBalance = 0;
41          CPUTokenBalance = 0;
42          myCoin = new Coin();
43          isPlaying = 0;
44          gamesWon = 0;
45          myRoundMove = 0;
46          CPURoundMove = 0;
47          roundSum = 0;
48          input = new Scanner(System.in);
49      }
```

```

50
51  /* Constructor: Game(double t1, double t2)--This is the
52  * overloaded version of the Odd/Even game constructor for
53  * Part 2 of PP #3: It requires two doubles as parameters,
54  * which are passed as initial thresholds for my CPU-based
55  * player objects, p1 and p2, respectively. */
56
57  public Game (double t1, double t2) {
58
59      p1 = new ComputerPlayer(t1);
60      p2 = new ComputerPlayer(t2);
61  }
62
63  /* Method: play()--This version of the play method is for Part 1
64  * of PP #3: It takes no parameters and just plays one
65  * interactive version of the game, unless the human player
66  * chooses to play again. */
67
68  public void play() {
69
70      System.out.println("Hi, do you want to play a game of " +
71      "Odd/Even versus the CPU?" + "Type 1 for yes or 0 for no. \n");
72
73      isPlaying = input.nextInt();
74
75      while(isPlaying != 0) {
76
77          System.out.println("Pick your move for this round: draw 1 " +
78          "or draw 2? \n");
79          myRoundMove = input.nextInt();
80
81          System.out.println("You chose to draw number " +
82          myRoundMove + ". \n");
83
84          myCoin.flip();
85          CPURoundMove += myCoin.tossResult() + 1;
86          System.out.println("The CPU chose to draw number " +
87          CPURoundMove + ". \n");
88
89          roundSum += myRoundMove + CPURoundMove;
90
91          if(roundSum % 2 == 0) {
92
93              System.out.println("Sorry, you lose " +
94              roundSum + " tokens to the CPU! \n");
95              CPUTokenBalance += roundSum;
96              p1TokenBalance -= roundSum;
97              myRoundMove *= 0;
98              CPURoundMove *= 0;
99              roundSum *= 0;
100          }
101          else {

```

```

102
103     System.out.println("Congratulations, you win " +
104         roundSum + " tokens for yourself! \n");
105     gamesWon++;
106     p1TokenBalance += roundSum;
107     CPUTokenBalance -= roundSum;
108     myRoundMove *= 0;
109     CPURoundMove *= 0;
110     roundSum *= 0;
111
112 }
113 System.out.println("Enter 1 to play again or " +
114     "0 to end the OddEven game. \n");
115
116     isPlaying = input.nextInt();
117 }
118 System.out.println("Final Score: " +
119     p1TokenBalance + ". \n");
120
121     System.out.println("Total games won: " +
122         gamesWon + ". \n");
123 }
124
125
126 /* Method: play(int games)--This is the overloaded version of the
127 play method for Part 2 of PP #3. It takes a single integer as an
128 input parameter: this is the number of "computer vs. computer" games
129 that should be played in the Simulation.java program file */
130
131 public void play (int games) {
132
133     int i = 0;
134
135     while(i < games) {
136
137         if (Math.random() > p1.t) { //Case 1, for Player One
138             p1.roundMove += 2;
139         }
140         else {
141             p1.roundMove += 1;
142         }
143
144         if (Math.random() > p2.t) { //Case 2, for Player Two
145             p2.roundMove += 1;
146         }
147         else {
148             p2.roundMove += 2;
149         }
150
151         roundSum += (p1.getRoundMove()) + (p2.getRoundMove());
152         //To determine who won and by how much.
153

```

```

154     if(roundSum % 2 == 0) { //Player Two wins!
155
156         p1.TokenBalance -= roundSum;
157         p2.TokenBalance += roundSum;
158
159         roundSum *= 0;    //Must reset my round parameters
160         p1.roundMove *= 0; //after each round is over.
161         p2.roundMove *= 0;
162         i++;
163     }
164     else {                //Player One wins!
165         p1.TokenBalance += roundSum;
166         p2.TokenBalance -= roundSum;
167
168         roundSum *= 0;    //Must reset my round parameters
169         p1.roundMove *= 0; //after each round is over.
170         p2.roundMove *= 0;
171         i++;
172     }
173 }
174 }
175
176 /* Method: getP1Score()--This method should return the current
177 score (number of tokens) that Player 1 has */
178
179 public int getP1Score() {
180
181     return p1.TokenBalance;
182 }
183
184 /* Method: getP2Score()--This method should return the current score
185 (number of tokens) that Player 2 has */
186
187 public int getP2Score() {
188
189     return p2.TokenBalance;
190 }
191
192
193 /* Method: resetP1Score() --This method is a mutator that resets P1's
194 score represented by taring the TokenBalance integer variable. */
195
196 public void resetP1Score() {
197
198     p1.TokenBalance *= 0;
199 }
200
201
202 /* Method: resetP2Score()--This method is a mutator that resets P2's score
203 represented by taring the TokenBalance integer variable. */
204
205 public void resetP2Score() {

```

```
206
207     p2.TokenBalance *= 0;
208
209 }
210 }
```

```
1  /* Name: Jinku Jung
2   * UNI: dj2598
3   * Assignment: Programming Project #3
4   * Date: 2/27/2021
5   */
6
7  /* File: Simulation.java
8   *
9   * Purpose: This is the Simulation class file for the Odd/Even game.
10  * Simulation.java can run simulations, of up to N games total, to determine:
11  *
12  * a.) Whether or not the Odd/Even game is fair in general and
13  * b.) If it's not fair, find out:
14  *     i.) Which player has a advantage clear advantage and
15  *     ii.) What will his dominant strategy be to maximize is overall
16  *          total winnings, no matter what the other player chooses to
17  *          throw.
18  */
19
20 public class Simulation{
21
22     public static void main(String[] args) {
23
24
25         double minThreshold = 0;    /* Represents a "damage-control" type
26                                     * threshold P2 uses in response to
27                                     * P1's dominant strategy. */
28
29         double maxThreshold = 0;    /* Represents the most optimal
30                                     * threshold P1 must use in order to
31                                     * always have a net gain or break
32                                     * even (for a large number of N
33                                     * games played) versus P2. */
34
35         int rowP1Score = 0;
36
37         for(double i = 0; i <= 1; i += 0.02) {
38
39             for(double j = 0; j <= 1; j += 0.02) {
40
41                 Game g = new Game(i, j); //Initializes the Game object 'g.'
42
43                 g.play(100000);    /* Plays 100,000 games, for a good
44                                     * sample size 'N.' */
45
46                 rowP1Score = g.getP1Score(); /* Updates rowP1Score variable
47                                     * per every set of games played
48                                     * (here, per every 100,000). */
49             }
46         }
47     }
48 }
49
```

```
50     if(j >= 0 && rowP1Score > 0 && i == 0) {
51         // Evaluates P2's best move, for
52         minThreshold = j;    // a given P1 threshold value.
53         maxThreshold = 1 - j; // Uses a "maxmin" algorithmic
54     }                        // concept, from which P1's
55     }                        // most optimal threshold value
56 }                            // can be deduced using basic
57 System.out.println(maxThreshold); // Probability & Game Theory.
58 System.out.println(minThreshold); // (see ReadMe.txt for more
59 }                             // info)
60 }
```

```

1  README.txt
2  Name: Jinku Jung
3  UNI: dj2598
4  Assignment: Programming Project #3
5
6  =====
7  Part 1: Single-player version of Odd/Even game.
8  =====
9
10 Creating this version of Odd/Even was pretty straight forward, based upon
11 Professor Cannon's Coin class file in lecture and my In-class Exercise 3 code
12 submission. Extending these code examples, I was able to successfully create
13 the Odd/Even game.
14
15
16 =====
17 Part 2: CPU vs. CPU version of Odd/Even game in Simulation mode.
18 =====
19 Given payoff matrix for Player One:
20
21           |   one finger thrown   |   two fingers thrown
22 -----
23 one finger thrown: |   -2 tokens lost   |   +3 tokens gained
24 two fingers thrown: |   +3 tokens gained   |   -4 tokens lost
25
26
27 Based on TA Melody's Game Theory themed bonus lecture, I broke out my Game
28 Theory notes from an old Economics class I took years ago . . .
29
30 From Player One's point of view: Suppose Player One calls "one finger" 3/5th's
31 of the time and "two fingers" 2/5th's of the time (at random).
32
33 In this case:
34
35 1. If Player Two calls 'one finger', Player One loses 2 tokens 3/5th's of
36 the time and wins 3 tokens 2/5th's of the time.
37
38 **FYI: On average, here Player One nets  $-2(3/5) + 3(2/5) = 0$  tokens/game.
39
40 2. If Player Two calls 'two fingers', Player One wins 3 tokens 3/5th's of
41 the time and loses 4 tokens 2/5th's of the time.
42
43 **FYI: On average, here Player One nets  $3(3/5) - 4(2/5) = 0.20$  tokens/game.
44
45 **In Other Words: If Player One mixes his moves thrown according to a certain
46 mixed strategy, Player One is guaranteed to break even--no matter what--as
47 long as Player Two strictly chooses to throw "one finger" in any given round.
48
49 If Player Two throws "two fingers" down in any given round--no matter

```

50 what--Player One is guaranteed to net 0.20 tokens/game.
51
52 CONCLUSION: By employing a simple mixed strategy tied to a particular
53 probability, Player One is guaranteed of at least breaking even, in the
54 worst-case scenario--no matter what move Player Two chooses to throw.
55
56 -----
57
58 Using basic Probability & math:
59
60 Let p denote the probability of times that Player One will throw "one
61 finger" during an Odd/Even game round. Choosing p , such that, Player One
62 wins the same amount per game--no matter what finger/fingers Player Two
63 chooses to throw.
64
65 We can also deduce the probability of Player One throwing "two fingers" as
66 $(1 - p)$.
67
68 Then, because:
69
70 a.) Player One's average winnings (when Player Two throws "one finger")
71 is $-2p + 3(1 - p)$ and
72
73 b.) Player One's average winnings (when Player Two throws "two fingers")
74 is $3p - 4(1 - p)$,
75
76 We can find an equilibrium solution by equating a.) to b.):
77
78 $-2p + 3(1 - p) = 3p - 4(1 - p)$
79 $3 - 5p = 7p - 4$
80 $12p = 7$
81 $p = 7/12$ or 0.583333 percent, a.k.a, 58.3333% percentage points
82
83 **NOTE: p should be the likelihood that Player One chooses to throw down
84 "one finger" in any given round; else, Player One is strictly better off
85 throwing "two fingers" at all other times, given his particular mixed
86 strategy.
87
88 Thus, Player One should call "one finger" 7/12th's of the time AND
89 "two fingers" 5/12th's of the time.
90
91 On the average, Player One will win:
92
93 "Average_Winnings" = $-2(7/12) + 3(5/12) = 1/12$, or 0.0833 tokens/game
94
95 **FYI: This is true for the total times he plays Odd/Even with Player
96 Two, given a large enough sample-size of N games.
97
98 **NOTE: This kind of mixed-strategy is called a Nash Equilibrium in
99 Economic Game Theory.
100
101 So . . . the game is clearly in Player One's favor. Can he do better

102 than 0.08333 tokens/game though?
103
104 NO: Not unless Player Two is an absolute idiot. In fact, Player Two
105 can use the same mixed strategy, but from his perspective:
106
107 Let Player Two throw "one finger" $7/12$'s of the time; else,
108 Player Two will throw "two fingers" $5/12$'s of the time.
109
110 CASE 1: When Player One throws "one finger" himself, Player Two's
111 average loss is:
112
113 "P2_Average_Losings" = $-2(7/12) + 3(5/12) = 1/12$ or
114 0.0833 tokens per game.
115
116 CASE 2: When Player One throws "two fingers" himself, Player Two's
117 average loss is:
118
119 "P2_Average_Losings" = $3(7/12) - 4(5/12) = 1/12$ or
120 0.0833 tokens per game.
121
122 **NOTE: 0.0833 tokens/game represents the VERY LEAST that Player
123 Two can lose, per game, when playing against Player One's most
124 optimal mixed strategy of $(p, 1-p) = (7/12, 5/12)$. Player
125 Two can actually lose MORE than 0.08333 tokens per game if he's
126 absolutely irrational while throwing fingers down per round!
127
128 So . . . $(p, 1-p)$ translates to (t_1, t_2) for my two-player
129 CPU vs CPU Odd/Even game simulation program.
130
131 Using a maxmin algorithmic technique, I get:
132
133 $p = \text{maxThreshold} = t_1 = 0.57999$ for Player One and
134
135 $(1 - p) = \text{minThreshold} = t_2 = 0.42000$ for Player Two.
136
137 **NOTE: I guess my calculations are slightly off due to a
138 floating-point rounding error or truncation. But, they are
139 within a very small margin of error. --Jinku Jung
